

Lecture Notes on Strings in C

Mohammad Maksood Akhter, PhD
Assistant Professor
Department of Computer Science and Engineering

Abstract

These notes introduce the concept of **strings in C programming**, their internal representation, and how they are manipulated in memory. The section includes detailed explanations, code examples, and memory intuitions for a strong conceptual understanding.

I. STRINGS IN C

In the C language, a **string** is a sequence of characters terminated by a special character called the **null character** (`\0`). Strings are stored in arrays of type `char`.

A. Definition and Basics

- A string in C is represented as a `char` array.
- Every string in C is terminated with a **null character** (`\0`), which has an ASCII value of 0.
- Anything enclosed within double quotes (" ") is treated as a **string constant**.

Example 1 – String as a Character Array

```
1 char s[10] = "INDIA";
```

Example 2 – String Constant with Pointer

```
1 char *s = "INDIA";
```

Both declarations store the same sequence of characters, but the first keeps them in an array (modifiable), and the second points to a constant string literal.

The **name of a string (array name)** always represents the *base address* of its first character in memory. Hence, in expressions like `printf("%s", str);`, the name `str` automatically provides the starting address for printing.

B. Examples and Intuition

Example 3 – Printing Individual Characters

```
1 #include <stdio.h>
2 void main() {
3     char str[20] = {'a', 'b', 'c', 'd', 'e', '\0'};
4     for(int i = 0; str[i] != '\0'; i++)
5         printf("%c ", str[i]);
6 }
```

Output: a b c d e

Each character in the string is stored in consecutive memory locations. The loop prints one character at a time until it reaches the null terminator `\0`.

Example 4 – Printing a String Using %s

```

1      #include <stdio.h>
2      void main() {
3          char str[20] = "abcd";
4          printf("%s", str);
5      }

```

Output: abcd

Intuition: When the string is printed using %s, printf() starts reading from the **base address** of str (say, 100). It continues printing characters sequentially until it encounters the null character (\0), which signals the end of the string.

C. Memory Representation and ASCII Values

Example 5 – Memory View of a String

```
char S[6] = "ABCD";
```

Memory Representation:

Index	0	1	2	3	4
Character	A	B	C	D	\0
ASCII	65	66	67	68	0

Each element is stored sequentially in memory. If S begins at address 100, then:

$$S[0] = 100, S[1] = 101, S[2] = 102, S[3] = 103, S[4] = 104$$

The string occupies 5 bytes in total (4 characters + 1 null terminator).

If you access $S[i]$, you are accessing the **character** at that position, which internally corresponds to its ASCII value. For example, $S[1]$ has value 66 because the ASCII code of 'B' is 66.

Key Takeaways

- Strings in C are arrays of characters ending with a null character (\0).
- The array name (string name) acts as a pointer to the first character.
- Using %s automatically prints characters until \0.
- Each character is stored sequentially in contiguous memory.

II. INITIALIZATION OF STRINGS IN C

A string in C can be initialized in multiple ways. The method of initialization determines whether the string is stored as a **modifiable character array** or as a **string constant** in read-only memory.

A. Different Ways to Initialize Strings

Examples:

```

1      char s[6] = "INDIA";           // Fixed-size array initialization
2      char s[] = "INDIA";           // Compiler auto-counts size (6 bytes including '\0')
3      char S[6] = {'I', 'N', 'D', 'I', 'A', '\0'}; // Explicit element-wise initialization
4      char S[] = {'I', 'N', 'D', 'I', 'A', '\0'}; // Size inferred automatically
5      char *p = "HELLO";           // String constant pointer

```

When you write:

```
char *p = "HELLO";
```

the compiler stores the string "HELLO" in read-only memory, and the pointer `p` holds the **base address** of that memory location.

B. Char Array vs String Constant

Aspect	Normal String (Char Array)	String Constant (Pointer)
Declaration	<code>char s[6] = "INDIA";</code>	<code>char *p = "INDIA";</code>
Storage	Stored in stack memory (modifiable)	Stored in read-only memory (string literal area)
Modifiability	Characters can be modified.	Modification not allowed — causes runtime error.
Example	<code>s[1] = 'A';</code> Allowed	<code>p[1] = 'A';</code> Not allowed
Base Address Behavior	Base address is fixed — <code>s++</code> or <code>s = 200;</code> not allowed.	Pointer can be reassigned — <code>p++</code> or <code>p = 200;</code> allowed.

C. Illustration through Examples

Example 1 – Character Array (Modifiable)

```
1 #include <stdio.h>
2 void main() {
3     char s[6] = "INDIA";
4     s[1] = 'O'; // Modifying the second character
5     printf("%s", s);
6 }
```

Output: IODIA

Example 2 – String Constant (Immutable)

```
1 #include <stdio.h>
2 void main() {
3     char *p = "INDIA";
4     p[1] = 'O'; //Not allowed causes segmentation fault
5     printf("%s", p);
6 }
```

Error: Attempt to modify a string literal stored in read-only memory.

D. Understanding Memory Allocation

When you declare:

```
char s[] = "INDIA";
```

Memory layout in stack (modifiable):

Index	0	1	2	3	4
Character	I	N	D	I	A
ASCII	73	78	68	73	65

A hidden sixth byte stores the null terminator `\0`.

Compiler Behavior:

- If you omit the size (e.g., `char s[] = "HELLO";`), the compiler automatically allocates space for all characters plus the null terminator.
- The string name (`s`) represents the starting address of the array.
- In pointer-based initialization (`char *p = "HELLO";`), only the address of the constant string is stored in `p`.

Key Insights

- Character arrays are **modifiable**, but their base addresses cannot be reassigned.
- String constants (`char *p`) point to **read-only memory**.
- Attempting to modify a string constant leads to a segmentation fault.
- Array size is always one more than the number of visible characters (to store `\0`).

III. ADVANCED EXAMPLES AND MEMORY BEHAVIOR OF STRINGS

A. Pointer and Array Assignments

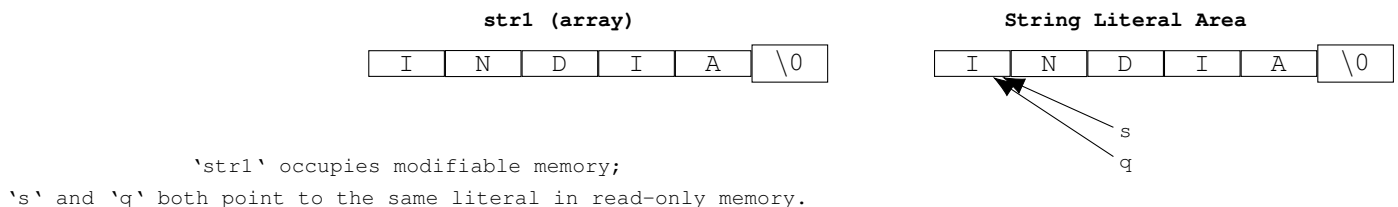
Example 1 – Pointer and Array Assignments

```

1  #include <stdio.h>
2
3  void main() {
4      char str1[] = "INDIA";
5      char str2[20];
6      char *s = "INDIA";
7      char *q;
8
9      // str2 = str1;    // Not allowed: cannot assign one array to another directly
10     q = s;             // Allowed: pointer q now points to same location as s
11
12     printf("String s: %s\n", s);
13     printf("String q: %s\n", q);
14 }

```

Explanation: In C, array names are constant addresses — you cannot assign one array to another directly. However, pointers like `s` and `q` can hold and copy addresses freely.

Graphical Representation:

Key Point: String literals are stored in read-only memory. Array variables (`str1`) store characters in stack memory (modifiable), while pointers (`s`, `q`) simply hold the address of a constant string.

B. Comparing Strings Using == Operator

Example 2 – Comparing Strings with ==

```
1      #include <stdio.h>
2
3      int main() {
4          char str1[10] = "hello";
5          char str2[10] = "hello";
6
7          if (str1 == str2) {
8              printf("Strings are equal\n");
9          } else {
10             printf("Strings are different\n");
11          }
12
13         return 0;
14     }
```

Output: Strings are different

Explanation: The operator == compares the **base addresses** of str1 and str2, not their contents. Although both contain “hello”, they are stored in different memory locations.

To compare content correctly, use:

```
1      if (strcmp(str1, str2) == 0)
2          printf("Strings are equal");
```

C. Dereferencing String Literals

Example 3 – Comparing First Characters Only

```
1      #include <stdio.h>
2
3      int main() {
4          if (*"hello" == *"hail") {
5              printf("They are same\n");
6          } else {
7              printf("They are different\n");
8          }
9          return 0;
10     }
```

Output: They are same

Explanation: *"hello" and *"hail" both dereference to their first character ‘h’. Since the ASCII value of ‘h’ is the same in both cases (104), the condition is true. Only the first characters are compared — the rest of the string is ignored.

D. Pointer Arithmetic and Null Terminator Effects

Example 4 – Pointer Arithmetic and String Modification

```

1  #include <stdio.h>
2
3  int main() {
4      char str[20] = "abcdef";
5
6      printf("%s\n", str);           // prints "abcdef"
7      printf("%s\n", str + 3);      // prints from index 3 "def"
8
9      str[3] = 97;                   // 97 is ASCII for 'a'
10     printf("%s\n", str);           // prints "abcaef"
11
12     str[3] = '\0';                 // insert null terminator
13     printf("%s\n", str);           // prints "abc"
14
15     str[3] = 0;                    // another way to insert null
16     printf("%s\n", str);           // prints "abc"
17
18     str[3] = 'd';                  // restore character
19     printf("%s\n", str);           // prints "abcd"
20
21     return 0;
22 }

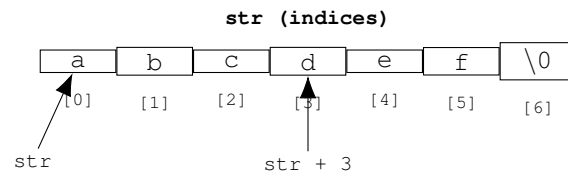
```

Output:

```

abcdef
def
abcaef
abc
abc
abcd

```



printf("%s", str+3); prints from index 3, output "def".
 Inserting str[3] = '\0'; truncates the string at that position.

Explanation:

- str + 3 points to the address of str[3].
- Printing from str + 3 starts output from that character onward.
- Assigning '\0' inserts a null terminator and ends the string.
- Replacing characters directly is allowed because str is stored in modifiable stack memory.

Key Learning Points:

- Arrays cannot be assigned directly (str2 = str1; not allowed).
- == compares addresses, not contents.
- Dereferencing string literals (*"abc") compares only the first character.
- Pointer arithmetic (str + n) accesses substrings from the nth index.
- Null terminator (\0) defines the logical end of a string.

IV. STRING FUNCTIONS IN C

The C standard library provides a collection of useful functions for manipulating strings. These functions are declared in the header file `<string.h>`. They handle common operations like finding length, copying, concatenation, and comparison.

A. Commonly Used String Functions

Function	Purpose	Prototype
<code>strlen()</code>	Finds the length of a string	<code>size_t strlen(const char *str);</code>
<code>strcpy()</code>	Copies one string into another	<code>char *strcpy(char *dest, const char *src);</code>
<code>strcat()</code>	Concatenates (joins) two strings	<code>char *strcat(char *dest, const char *src);</code>
<code>strcmp()</code>	Compares two strings	<code>int strcmp(const char *s1, const char *s2);</code>
<code>strncpy()</code>	Copies <i>n</i> characters from source to destination	<code>char *strncpy(char *dest, const char *src, size_t n);</code>
<code>strncat()</code>	Concatenates <i>n</i> characters from source to destination	<code>char *strncat(char *dest, const char *src, size_t n);</code>
<code>strchr()</code>	Finds first occurrence of a character	<code>char *strchr(const char *str, int c);</code>
<code>strstr()</code>	Finds first occurrence of a substring	<code>char *strstr(const char *str, const char *substr);</code>

B. 1. `strlen()` – String Length

Example: Find the Length of a String

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char name[] = "SOA University";
6      int len = strlen(name);
7
8      printf("String: %s\n", name);
9      printf("Length: %d\n", len);
10     return 0;
11 }
```

Output:

```
String: SOA University
Length: 14
```

Note: `strlen()` counts characters until the null terminator `\0`, but does not include it in the count.

C. 2. strcpy() – Copying Strings

Example: Copy One String into Another

```
1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char src[] = "C Programming";
6          char dest[50];
7
8          strcpy(dest, src);
9
10         printf("Source: %s\n", src);
11         printf("Destination: %s\n", dest);
12         return 0;
13     }
```

Output:

```
Source: C Programming
Destination: C Programming
```

Important: Ensure the destination array is large enough to hold the source string (including the null terminator) to avoid buffer overflow.

D. 3. strcat() – Concatenating Strings

Example: Joining Two Strings

```
1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char first[50] = "Hello ";
6          char second[] = "World!";
7
8          strcat(first, second);
9
10         printf("After concatenation: %s\n", first);
11         return 0;
12     }
```

Output:

```
After concatenation: Hello World!
```

Explanation: `strcat(dest, src)` appends the contents of `src` to `dest` and automatically adds the null terminator at the end.

E. 4. strcmp() – Comparing Strings

Example: Comparing Two Strings

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char a[] = "apple";
6          char b[] = "banana";
7          char c[] = "apple";
8
9          printf("strcmp(a, b) = %d\n", strcmp(a, b));
10         printf("strcmp(a, c) = %d\n", strcmp(a, c));
11         printf("strcmp(b, a) = %d\n", strcmp(b, a));
12         return 0;
13     }

```

Output:

```

strcmp(a, b) = -1
strcmp(a, c) = 0
strcmp(b, a) = 1

```

Explanation:

- Returns 0 if both strings are equal.
- Returns <0 if first string is lexicographically smaller.
- Returns >0 if first string is lexicographically greater.

F. 5. sizeof vs strlen

Example: Difference Between sizeof and strlen

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char str1[] = "ITER";
6          char str2[20] = "ITER";
7
8          printf("sizeof(str1) = %lu\n", sizeof(str1));
9          printf("strlen(str1) = %lu\n", strlen(str1));
10
11         printf("sizeof(str2) = %lu\n", sizeof(str2));
12         printf("strlen(str2) = %lu\n", strlen(str2));
13
14         return 0;
15     }

```

Output:

```

sizeof(str1) = 5
strlen(str1) = 4
sizeof(str2) = 20
strlen(str2) = 4

```

Key Difference:

- sizeof() returns the **total memory allocated** for the array (including \0 and unused space).
- strlen() returns the **actual number of characters** before the null terminator.

G. 6. `strchr()` and `strstr()`

Example: Searching Within Strings

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char text[] = "C programming language";
6          char *p1 = strchr(text, 'p');
7          char *p2 = strstr(text, "language");
8
9          printf("First occurrence of 'p': %s\n", p1);
10         printf("Substring found: %s\n", p2);
11         return 0;
12     }

```

Output:

```

First occurrence of 'p': programming language
Substring found: language

```

`strchr()` returns a pointer to the first occurrence of a character. `strstr()` returns a pointer to the first occurrence of a substring. If not found, both return `NULL`.

Summary of Best Practices

- Always include `<string.h>` when using string functions.
- Ensure destination arrays are large enough for copy/concatenation operations.
- Use `strcmp()` for comparisons, not `==`.
- Remember: `sizeof()` gives total size, while `strlen()` gives actual length.
- Always check for `NULL` when using `strchr()` or `strstr()`.

V. INTRODUCTION TO STRINGS AND MULTIDIMENSIONAL STRINGS

Strings are sequences of characters stored as arrays in C. Beyond single-dimensional strings, C also supports **multidimensional strings** — arrays of strings — which are useful for storing lists of words or sentences.

A. Single-Dimensional vs Multidimensional Strings

- **Single-Dimensional String:** A normal character array terminated with a null character.
- **Multidimensional String:** An array of character arrays, each representing a separate string.

Example: Single-Dimensional String

```

1      char str[20] = "Hello World";

```

Example: Multidimensional Strings (Array of Strings)

```

1      char names[3][20] = {"Alice", "Bob", "Charlie"};

```

Accessing elements: `names[0]` refers to "Alice", `names[1][2]` refers to the 3rd character of "Bob" (i.e., 'b').

B. Managing Strings: Palindrome Check

Example: Check if a String is a Palindrome

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char str[50];
6          int len, flag = 1;
7
8          printf("Enter a string: ");
9          scanf("%s", str);
10
11         len = strlen(str);
12         for(int i = 0; i < len/2; i++) {
13             if(str[i] != str[len-1-i]) {
14                 flag = 0;
15                 break;
16             }
17         }
18
19         if(flag)
20             printf("Palindrome\n");
21         else
22             printf("Not a Palindrome\n");
23
24         return 0;
25     }

```

Explanation: Compare characters from start and end moving toward the center.

VI. STRING MANIPULATION PROBLEMS

A. 1. Determine Whether a String is a Palindrome

Problem: Check if a given string reads the same forwards and backwards.

Example Code:

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char str[50];
6          int len, flag = 1;
7
8          printf("Enter a string: ");
9          scanf("%s", str);
10
11         len = strlen(str);
12         for(int i = 0; i < len/2; i++) {
13             if(str[i] != str[len-1-i]) {
14                 flag = 0;
15                 break;
16             }
17         }
18
19         if(flag)
20             printf("Palindrome\n");
21         else
22             printf("Not a Palindrome\n");
23
24         return 0;
25     }

```

B. 2. Find the Occurrence of a Character in a String

Problem: Find the first occurrence of a given character in a string.

Example Code:

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char str[50], ch;
6          int pos = -1;
7
8          printf("Enter a string: ");
9          scanf("%s", str);
10         printf("Enter character to search: ");
11         scanf(" %c", &ch);
12
13         for(int i = 0; str[i] != '\0'; i++) {
14             if(str[i] == ch) {
15                 pos = i;
16                 break;
17             }
18         }
19
20         if(pos != -1)
21             printf("Character found at position %d\n", pos);
22         else
23             printf("Character not found\n");
24
25         return 0;
26     }

```

C. 3. Display Count for Each Character (Manual Counting)

Problem: Count frequency of each character manually without extra arrays.

Example Code:

```

1      #include <stdio.h>
2      #include <string.h>
3
4      int main() {
5          char str[50];
6          printf("Enter a string: ");
7          scanf("%s", str);
8
9          int len = strlen(str);
10
11         printf("Character frequencies:\n");
12         for(int i = 0; i < len; i++) {
13             int count = 1;
14             if(str[i] == 0) continue; // Already counted
15             for(int j = i+1; j < len; j++) {
16                 if(str[i] == str[j]) {
17                     count++;
18                     str[j] = 0; // Mark as counted
19                 }
20             }
21             printf("%c: %d\n", str[i], count);
22         }
23
24         return 0;
25     }

```

D. 4. Count Vowels and Consonants in a Sentence

Problem: Count vowels and consonants manually using ASCII logic.

Example Code:

```

1  #include <stdio.h>
2
3  int main() {
4      char str[100];
5      int vowels = 0, consonants = 0;
6
7      printf("Enter a string: ");
8      scanf("%s", str);
9
10     for(int i = 0; str[i] != '\0'; i++) {
11         char ch = str[i];
12         if(ch >= 'A' && ch <= 'Z') ch += 32; // Convert to lowercase
13         if(ch >= 'a' && ch <= 'z') {
14             if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
15                 vowels++;
16             else
17                 consonants++;
18         }
19     }
20
21     printf("Vowels: %d\n", vowels);
22     printf("Consonants: %d\n", consonants);
23
24     return 0;
25 }

```

E. 5. Convert Vowels in a Sentence to Uppercase and Lowercase (ASCII A = 65 and a = 97, Difference is 32)

Problem: Convert vowels using ASCII arithmetic.

Example Code:

```

1  #include <stdio.h>
2
3  int main() {
4      char str[100];
5
6      printf("Enter a string: ");
7      scanf("%s", str);
8
9      for(int i = 0; str[i] != '\0'; i++) {
10         char ch = str[i];
11
12         // Lowercase vowels -> uppercase
13         if(ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u')
14             str[i] = ch - 32;
15
16         // Uppercase vowels -> lowercase
17         if(ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U')
18             str[i] = ch + 32;
19     }
20
21     printf("Modified string: %s\n", str);
22
23     return 0;
24 }

```

VII. LAB EXERCISES ON STRINGS

A. LAB 007

Programming Tasks:

- Display the count of each character in a string.
- Count vowels and consonants in a sentence.

*B. LAB 008***Programming Tasks:**

- Convert all vowels in a sentence to uppercase.
- Find the occurrence of the first repetitive character in a string.

END OF NOTES